

EXPERIMENT 9

IMPLEMENTATION OF RC5 ALGORITHM

AIM:

To implement the RC5 (Rivest Cipher 5) symmetric key encryption algorithm in C and perform encryption and decryption of a plaintext block using a secret key.

ALGORITHM:

Key Expansion

1. Initialize the RC5 parameters:
 - Word size $w = 32$ bits
 - Number of rounds $r = 12$
 - Key length = 16 bytes
2. Initialize the expanded key array S .
3. Mix the user-provided secret key with the S array.
4. Generate the round keys required for encryption and decryption.

Encryption:

1. Divide the plaintext into two 32-bit words A and B .
2. Add the first two subkeys:

$$A = A + S[0]$$

$$B = B + S[1]$$

3. Perform 12 rounds. In each round:

$$A = ((A \text{ XOR } B) \lll B) + S[2i]$$

$$B = ((B \text{ XOR } A) \lll A) + S[2i+1]$$

where $\lll$ represents left circular rotation.

4. Combine A and B to obtain the ciphertext.

Decryption:

1. Divide the ciphertext into A and B .
2. Perform the 12 rounds in reverse order.
3. Apply right circular rotations.
4. Subtract the corresponding round keys.
5. Finally subtract $S[1]$ from B and $S[0]$ from A .

6. The original plaintext is obtained.

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
#include <string.h>
```

```
#define ROUNDS 12
```

```
#define S_SIZE (2 * ROUNDS + 2)
```

```
#define P32 0xB7E15163
```

```
#define Q32 0x9E3779B9
```

```
uint32_t S[S_SIZE];
```

```
/* Left circular rotation */
```

```
uint32_t ROTL(uint32_t x, uint32_t y)
```

```
{
```

```
    y &= 31;
```

```
    if (y == 0)
```

```
        return x;
```

```
    return (x << y) | (x >> (32 - y));
```

```
}
```

```
/* Right circular rotation */
```

```
uint32_t ROTR(uint32_t x, uint32_t y)
```

```
{
```

```
    y &= 31;
```

```
    if (y == 0)
```

```
        return x;
```

```
    return (x >> y) | (x << (32 - y));
```

```
}
```

```
/* RC5 Key Expansion */
```

```
void RC5_KeyExpansion(unsigned char *key, int keyLen)
```

```
{
```

```
    uint32_t L[4] = {0};
```

```
    uint32_t A, B;
```

```
    int i, j, k;
```

```
/*
```

```
 * Convert key bytes into 32-bit words.
```

```
*/
```

```
for (i = keyLen - 1; i >= 0; i--)
```

```
{
```

```
    L[i / 4] =
```

```
        (L[i / 4] << 8) | key[i];
```

```
}
```

```
/* Initialize S array */
```

```
S[0] = P32;
```

```
for (i = 1; i < S_SIZE; i++)
```

```
{
```

```
    S[i] = S[i - 1] + Q32;
```

```
}
```

```
/*
```

```
 * Mix S and L arrays.
```

```
*/
```

```
A = 0;
```

```
B = 0;
```

```
i = 0;
```

```
j = 0;
```

```

for (k = 0; k < 3 * S_SIZE; k++)

{

    A = S[i] =

        ROTL(S[i] + A + B, 3);


    B = L[j] =

        ROTL(L[j] + A + B,

            A + B);


    i = (i + 1) % S_SIZE;

    j = (j + 1) % 4;

}

}

/* RC5 Encryption */

void RC5_Encrypt(uint32_t *A, uint32_t *B)

{

    int i;


    *A = *A + S[0];

    *B = *B + S[1];


    for (i = 1; i <= ROUNDS; i++)

```

```

{
    *A =

    ROTL(*A ^ *B, *B)

    + S[2 * i];

    *B =

    ROTL(*B ^ *A, *A)

    + S[2 * i + 1];

}

}

/* RC5 Decryption */

void RC5_Decrypt(uint32_t *A, uint32_t *B)

{
    int i;

    for (i = ROUNDS; i >= 1; i--)

    {
        *B =

        ROTR(*B - S[2 * i + 1],

        *A) ^ *A;

        *A =

```

```

        ROTR(*A - S[2 * i],

            *B) ^ *B;

    }

    *B = *B - S[1];

    *A = *A - S[0];

}

int main()

{

    uint32_t A, B;

    uint32_t encryptedA, encryptedB;

    uint32_t decryptedA, decryptedB;


    unsigned char key[16];


    printf("RC5 ALGORITHM\n");

    printf("-----\n");


    printf("Enter 16-digit hexadecimal plaintext: ");

    scanf("%8x%8x", &A, &B);


    printf("Enter 16-character key: ");

```

```
scanf("%16s", key);

/* Key expansion */

RC5_KeyExpansion(key, strlen((char *)key));

/* Store plaintext */

encryptedA = A;

encryptedB = B;

/* Encryption */

RC5_Encrypt(&encryptedA, &encryptedB);

printf("\n\nENCRYPTION\n");

printf("-----\n");

printf("Plaintext : %08X%08X\n",

      A, B);

printf("Ciphertext : %08X%08X\n",

      encryptedA,

      encryptedB);

/* Decryption */
```



```
decryptedA = encryptedA;
```

```
decryptedB = encryptedB;
```

```
RC5_Decrypt(&decryptedA, &decryptedB);
```

```
printf("\nDECRYPTION\n");
```

```
printf("-----\n");
```

```
printf("Ciphertext : %08X%08X\n",
```

```
    encryptedA,
```

```
    encryptedB);
```

```
printf("Plaintext : %08X%08X\n",
```

```
    decryptedA,
```

```
    decryptedB);
```

```
if (decryptedA == A &&
```

```
    decryptedB == B)
```

```
{
```

```
    printf("\nEncryption and Decryption Successful!\n");
```

```
}
```

```
else
```

```
{
```

```

printf("\nEncryption and Decryption Failed!\n");

}

return 0;

}

```

OUTPUT:

```

main.c
164
165 printf("\nDECRYPTION\n");
166 printf("-----\n");
167
168 printf("Ciphertext : %08X%08X\n",
169        encryptedA,
170        encryptedB);
171
172 printf("Plaintext : %08X%08X\n",
173        decryptedA,
174        decryptedB);
175
176 if (decryptedA == A &&
177     decryptedB == B)
178 {
179     printf("\nEncryption and Decryption Successful!\n");
180 }
181 else
182 {
183     printf("\nEncryption and Decryption Failed!\n");
184 }
185
186 return 0;
187 }

```

```

RC5 ALGORITHM
-----
Enter 16-digit hexadecimal plaintext: 123456789ABCDEF0
Enter 16-character key: MYSECRETKEY123456

ENCRIPTION
-----
Plaintext : 123456789ABCDEF0
Ciphertext : 31BA21CEE780CEC9

DECRYPTION
-----
Ciphertext : 31BA21CEE780CEC9
Plaintext : 123456789ABCDEF0

Encryption and Decryption Successful!

*** Code Execution Successful ***

```

RESULT:

Thus, the RC5 symmetric key encryption algorithm was successfully implemented in C. The given 64-bit plaintext was encrypted using the secret key, and the resulting ciphertext was successfully decrypted to recover the original plaintext.